ASSIGNMENT 7

SAMPLE DATA INSERTION

Project: Library Management System | Database: LibraryManagementSystem

Student Name: _____ **Class:** _____

1. Introduction

The purpose of this assignment is to insert realistic sample records into all nine tables of the Library Management System. The sample data represents authors, publishers, categories, books, members, staff, loans, reservations and fines. The records are designed to follow the primary-key, foreign-key and UNIQUE constraints in the submitted database schema.

2. Important Notes

• Run the original database creation script first so that all nine tables exist.
• Select the database using USE LibraryManagementSystem; before inserting records.
• Insert parent-table records before child-table records to satisfy foreign-key relationships.
• The sample data uses the same book, member, staff, loan and fine IDs referenced in Assignments 8 and 9, so the example query outputs can be reproduced.
• If you have already inserted records with these IDs, clear the existing sample rows or use an empty database before running these INSERT statements.

3. SQL INSERT Statements

Step 1: Select the database

```
USE LibraryManagementSystem;
```

Step 2: Insert records into Authors

```
INSERT INTO Authors (AuthorID, AuthorName, Nationality, BirthYear) VALUES
(1, 'R. Elmasri', 'American', 1949),
(2, 'Mark Lutz', 'American', 1955),
(3, 'Stuart Russell', 'British', 1962),
(4, 'Andrew S. Tanenbaum', 'Dutch', 1944),
(5, 'Herbert Schildt', 'American', 1951);
```

Step 3: Insert records into Publishers

```
INSERT INTO Publishers (PublisherID, PublisherName, Address, Phone) VALUES
(1, 'Pearson Education', 'London, UK', '+442071234501'),
(2, 'OReilly Media', 'Sebastopol, USA', '+17078271200'),
(3, 'Prentice Hall', 'New Jersey, USA', '+18005550103'),
(4, 'McGraw Hill', 'New York, USA', '+12125550104');
```

Step 4: Insert records into Categories

```
INSERT INTO Categories (CategoryID, CategoryName, Description) VALUES
(1, 'Database', 'Books related to database concepts and design'),
(2, 'Programming', 'Books about programming languages and software'),
(3, 'Computer Science', 'Books covering core computer science subjects');
```

Step 5: Insert records into Books

```
INSERT INTO Books
(BookID, Title, ISBN, AuthorID, PublisherID, CategoryID,
 PublicationYear, TotalCopies, AvailableCopies)
VALUES
(101, 'Database Management Systems', '9780136930155', 1, 1, 1, 2016, 5, 3),
(102, 'Python Programming', '9781491939369', 2, 2, 2, 2013, 4, 2),
(103, 'Artificial Intelligence', '9780134610993', 3, 3, 3, 2020, 3, 1),
(104, 'Computer Networks', '9780132126953', 4, 1, 3, 2011, 6, 4),
(105, 'Java Programming', '9781260440232', 5, 4, 2, 2018, 5, 3);
```

Step 6: Insert records into Members

```
INSERT INTO Members
(MemberID, FullName, Email, Phone, Address, MembershipDate, MembershipType)
VALUES
(201, 'Asha Sharma', 'asha.sharma@example.com', '9876501201',
```

```
 'Bengaluru', '2026-01-10', 'Regular'),
(202, 'Rahul Verma', 'rahul.verma@example.com', '9876501202',
 'Anekal', '2026-02-15', 'Student'),
(203, 'Priya Nair', 'priya.nair@example.com', '9876501203',
 'Electronic City', '2026-03-05', 'Regular'),
(204, 'Karan Rao', 'karan.rao@example.com', '9876501204',
 'Bommasandra', '2026-04-12', 'Student');
```

Step 7: Insert records into Staff

```
INSERT INTO Staff
(StaffID, FullName, Role, Email, Phone, HireDate)
VALUES
(301, 'Meera', 'Librarian', 'meera.staff@example.com',
 '9876501301', '2023-06-01'),
(302, 'Arjun', 'Assistant Librarian', 'arjun.staff@example.com',
 '9876501302', '2024-02-15');
```

Step 8: Insert records into Loans

```
INSERT INTO Loans
(LoanID, BookID, MemberID, StaffID, IssueDate, DueDate, ReturnDate, Status)
VALUES
(1001, 101, 201, 301, '2026-10-01', '2026-10-15', '2026-10-12', 'Returned'),
(1002, 102, 202, 301, '2026-10-05', '2026-10-19', NULL, 'Issued'),
(1003, 103, 201, 302, '2026-09-20', '2026-10-04', '2026-10-08', 'Returned'),
(1004, 101, 203, 302, '2026-10-10', '2026-10-24', NULL, 'Issued'),
(1005, 105, 204, 301, '2026-09-25', '2026-10-09', '2026-10-13', 'Returned'),
(1006, 104, 202, 302, '2026-10-12', '2026-10-26', NULL, 'Issued');
```

Step 9: Insert records into Reservations

```
INSERT INTO Reservations
(ReservationID, BookID, MemberID, ReservationDate, Status)
VALUES
(6001, 103, 203, '2026-10-20', 'Pending'),
(6002, 101, 204, '2026-10-21', 'Confirmed'),
(6003, 102, 201, '2026-10-22', 'Pending');
```

Step 10: Insert records into Fines

```
INSERT INTO Fines
(FineID, LoanID, Amount, PaidStatus, PaidDate)
VALUES
(5001, 1003, 20.00, 'Paid', '2026-10-10'),
(5002, 1005, 15.00, 'Unpaid', NULL);
```

4. Verification Queries

After inserting the sample records, execute the following queries to confirm that data has been inserted into all nine tables.

```
SELECT COUNT(*) AS TotalAuthors FROM Authors;
SELECT COUNT(*) AS TotalPublishers FROM Publishers;
SELECT COUNT(*) AS TotalCategories FROM Categories;
SELECT COUNT(*) AS TotalBooks FROM Books;
SELECT COUNT(*) AS TotalMembers FROM Members;
SELECT COUNT(*) AS TotalStaff FROM Staff;
SELECT COUNT(*) AS TotalLoans FROM Loans;
SELECT COUNT(*) AS TotalReservations FROM Reservations;
SELECT COUNT(*) AS TotalFines FROM Fines;
```

Expected row counts

Table	Expected records
Authors	5
Publishers	4
Categories	3
Books	5
Members	4
Staff	2
Loans	6
Reservations	3
Fines	2

5. Verify Table Relationships

The following query checks that each loan is connected to an existing book, member and staff member.

```
SELECT
    L.LoanID,
    B.Title,
    M.FullName AS MemberName,
    S.FullName AS StaffName,
    L.Status
FROM Loans L
INNER JOIN Books B ON L.BookID = B.BookID
INNER JOIN Members M ON L.MemberID = M.MemberID
INNER JOIN Staff S ON L.StaffID = S.StaffID
ORDER BY L.LoanID;
```

Expected result: six rows should be returned, one for each loan record. Each row should display a valid book title, member name and staff name.

6. Conclusion

Sample records have been prepared for all nine tables of the Library Management System. The INSERT statements follow the order of table dependencies so that referenced authors, publishers, categories, books, members and staff exist before related transactions are inserted. The verification queries help confirm that records are present and that the relationships between tables work correctly.